

The problem: the page title never reaches the model

Hebrew RAG over Kol-Zchut · retrieval stage only · 24,487 paragraphs from 7,007 pages



One page in the data: a row per paragraph

הוצאת דרכון זמני
(temporary passport)

title: stored, not embedded

Paragraph 1: intro, one-year passport, fast track

Paragraph 2: who is eligible, preconditions

Paragraph 3: issuing at the offices

Paragraph 4: within 48 hours of the flight

Paragraph 5: contact block (repeated on thousands of pages)

The config embeds the content field only. 64% of paragraphs already open with their section heading, so the section is in. What is missing is the page title.

Observation: a held-out question

"דרכון דחוף" (urgent passport)

- 1 הוצאת דרכון ביומטרי (biometric)
- 2 הוצאת דרכון זמני (temporary): correct

The two bodies are nearly identical. The difference, temporary vs biometric, is **only in the title**.

Another pair: municipal-tax discount for the disabled vs for the work-disabled. Similarity of their opening paragraphs: 0.905.

Starting point, 296 questions

37%

right page first

58%

among the 3 pages sent to GPT

Of 187 misses, **63 at rank 2–3**, mostly behind a sibling page. The pattern above.

Another 71 are questions that do not identify a page on their own. That is the ceiling.

The fix: the title becomes part of the embedded text

One config key · the model is not retrained · search, API and frontend unchanged

Before

נכים עשויים לקבל מהרשות המקומית הנחה
בתשלומי הארנונה בהתאם לתנאים...



After

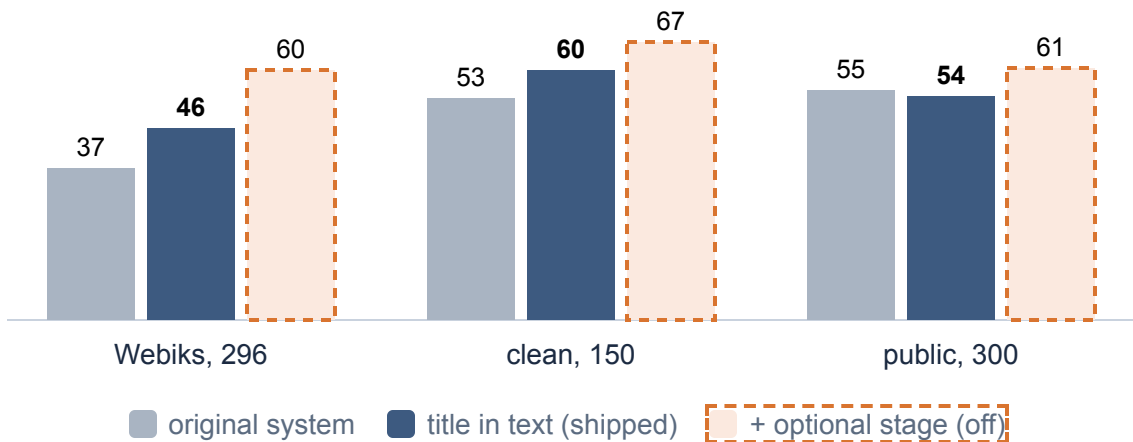
הנחה בארנונה לנכים
נכים עשויים לקבל מהרשות המקומית הנחה בתשלומי
הארנונה...



One vector per paragraph

As before, same field and layout. Its content now knows which page it belongs to.

Right page at rank 1, three question sets (%)



How it was measured

Webiks: their training code's validation split. **Clean**: 150 pages with no question in the QA file, one question each, written with the title hidden from the writer. **Public**: the 300 questions a competitor published. My evaluator matches Webiks' to 4 decimals.

Per question

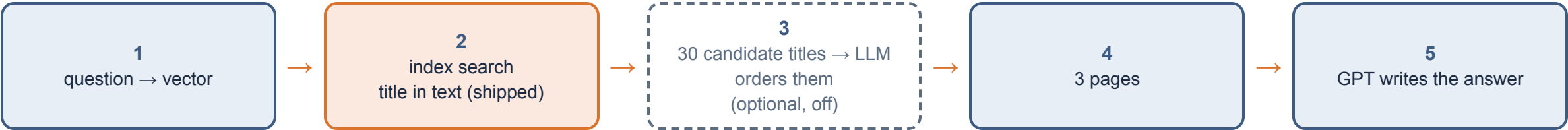
Webiks: 114 better, 47 worse; top 3 from 58% to 65%. Clean: 38 vs 19. On the public set the difference is noise: the title helps when a question names the population or sub-case.

Tested and switched off: a separate title vector at 0.3

53% on the Webiks set, but it gave the gain back on both independent sets. Off, kept as an option.

Optional second stage, off by default, and how it all fits

An LLM reorders the candidate pages by their titles · any provider through an LLM factory · one line to turn on



LLM factory: any model you want

OpenAI

Azure

Ollama

vLLM

Anthropic

Gemini

+ your own

One interface: messages in, text out. A new model is a small class registered by name.

☐ Off by default. To turn on, in `app/.env`:

```
TITLE_RERANK_ENABLED=true
TITLE_RERANK_PROVIDER=openai
TITLE_RERANK_MODEL=gpt-4o-mini
```

On any failure it falls back to the plain order. Cost: ~0.8 s per query and an LLM key; no effect in mock mode. Hence off.

Tested and rejected, each against the same setup without it

Experiment	Δ rank 1 / top 3, pp
256-token chunks + title	-5.8 / +2.7
LLM-written context per paragraph	-3.7 / -0.6
Query rewrite / HyDE / 4 paraphrases (best)	+2.4 / +3.8
Boilerplate filter / 500 candidates	±1.5 / 0

Rewriting: a small gain with heavy churn both ways, and an LLM in the search path.

Integration

60 tests pass, 19 of them new. Back to the original: remove one key and re-index. Found and fixed on the way: the fast indexing script ignored the config file.